

Homily Project Report

Overview

Homiely is a roommate coordination web application built with a React frontend, a Flask backend, and a PostgreSQL database. The application helps roommates coordinate shared chores, expenses, calendars, profiles, login state, and group membership. My work was mainly focused on the parts of the project that connected the system together: login routing, Home page state, group setup, profile membership behavior, Docker setup, shared test users, refactoring, and our Git workflow. In many ways, my role was not only to build individual pages, but also to make sure the different parts of the app could actually work together as one product.

Design Decisions and Alternatives Considered

1. Dockerized Multi-Service Development Environment

Initially, it seemed reasonable for each teammate to run the frontend, backend, and database locally. This would have meant starting React with npm, running Flask separately, and making sure PostgreSQL was installed and configured correctly on each machine. In theory, this was straightforward. However, in practice, this approach quickly became fragile. We encountered issues frequently at the beginning of development, particularly with Python version mismatches and environment configuration problems across different machines with Emma and Jerry. When something broke, it was often unclear whether the problem was in our code or only in one person's local setup. Managing local dependencies also became increasingly difficult as the project grew. As frontend, backend, and database interactions became more tightly coupled and diagnosing environment-specific bugs took way too long to debug. Because Homiely relies on reliable communication between React, Flask, and Postgres, we decided that a shared containerized environment would provide more value than local automation scripts alone.

Because of this, we chose to use Docker Compose. Docker allowed us to run the frontend, backend, and PostgreSQL database as separate services while still giving the team one shared development environment. This was important because Homiely depended on communication between all three layers: React sent asynchronous fetch requests, Flask validated JWTs and handled API logic, and PostgreSQL stored durable user, group, chore, expense, and availability data. By containerizing the services, we made integration bugs easier to reproduce because everyone was running the same architecture with the same dependencies and networking behavior.

The main alternative was to write setup scripts or Makefile commands for local development. This is an improvement that we overlooked and could have explored further as we could have had one script install dependencies and another script launch the frontend and backend together. This would have helped, and looking back, we probably should have still added a Makefile. However, local scripts still depend on each developer's machine being configured correctly. Docker reduced that dependency much more strongly, providing a similar level of convenience with only required commands such as docker

compose up and docker compose down -v. The tradeoff was that every teammate had to install and learn Docker, but that cost was small compared to the time we saved debugging environment-specific problems. Overall, Docker acted like executable documentation for our architecture.

This design decision tied directly to lecture concepts involving build automation, reproducibility, and reducing hidden assumptions between developers. Docker effectively served as executable documentation because the Compose file described the application's architecture, such as ports, environment variables, and networking behavior.

2. Choosing Postgres and a Relational Data Model

At the beginning of the project, we knew Homiely would need persistent storage for users, roommate groups, chores, expenses, and availability schedules. Initially, we focused more on getting authentication and basic frontend functionality working than on optimizing database design. Because of this, several database options were discussed while the project architecture was still evolving.

One alternative we considered was MongoDB. At first glance, MongoDB appeared attractive because it offers a flexible schema and allows developers to store data without defining strict relationships ahead of time. This flexibility seemed useful early in development because we were still determining exactly what information needed to be stored for groups, chores, and expenses.

As development progressed, however, we realized that Homiely's data was naturally relational. Almost every major feature depended on relationships between different entities. Users belong to roommate groups, groups contain multiple members, chores are assigned to specific users, expenses are created by one user but shared among multiple roommates, and availability schedules are tied to both users and groups. Because these relationships became more central to the project over time, a relational database increasingly felt like the correct abstraction for the problem we were solving.

Around the middle of development, when features such as group management, chore assignments, and expense tracking were being integrated together, the advantages of PostgreSQL became much more apparent. Rather than manually managing relationships inside application code, we could use tables, primary keys, and foreign keys to represent them directly within the database. This reduced the amount of custom logic needed in Flask and helped ensure data consistency. For example, expense records could be linked directly to users and groups, and chore assignments could reference specific users without requiring additional bookkeeping in the backend.

PostgreSQL also integrated naturally with our Flask backend. Flask handled API routes and business logic, such as authentication, group creation, joining groups, adding chores, and retrieving expenses, while PostgreSQL handled storage and relationship management. This separation of responsibilities resulted in a cleaner architecture where React communicated with Flask through API calls, Flask processed requests and enforced application logic, and PostgreSQL maintained the underlying data model.

Looking back, the primary tradeoff was between flexibility and structure. MongoDB would have provided greater schema flexibility early on. However, as the project grew and more features interacted with one another, the structure provided by PostgreSQL became increasingly valuable. Because Homiely's core functionality revolves around connected data and relationships between users, groups, chores, expenses, and schedules, PostgreSQL ultimately reduced complexity rather than adding to it.

The final design aligned well with the application's data model and allowed us to scale features without continuously redesigning how information was stored.

3. Group-Aware Routing and Protected Navigation

Another design decision I spent a lot of time thinking about was routing. The app had two different kinds of state: whether the user was logged in, and whether the user belonged to a roommate group. These sound similar, but they are different. A logged-out user should not be able to access private pages at all. A logged-in user without a group, however, is still in a valid state and needs to be directed toward group setup.

In commit 2ae75f5, I made login branch into two cases. If the user already had a roommate group, they were sent to /home. If they did not have a group yet, they were sent to /group-setup. On the Home page, I also used conditional rendering so grouped users saw real dashboard data, while ungrouped users saw an empty state telling them to create or join a group. This made the app feel more intentional because login did not simply mean "go to the dashboard"; it meant "go to the page that makes sense for this user."

For authentication, I wanted something stronger than page-by-page checks. A logged-out user should not be able to manually type /home, /chores, /expenses, /calendar, or /profile and access private household data. That led me to push for guarded routes, which Emma later implemented with ProtectedRoutes and LoggedInProtectedRoutes using AuthContext.jsx. This connected directly to the security lecture because authentication is not just a login form; it is also controlling access afterward.

Looking back, I think using both approaches was reasonable. Route guards handled the hard security boundary of whether the user was logged in, while conditional rendering handled the product state of whether the user had a group. However, as pages like Chores, Expenses, and Calendar also became dependent on group membership, we probably should have created a shared GroupContext or GroupProtectedRoute. That would have reduced repeated logic and made group-based access easier to reason about.

4. Shared Test Users and Test Groups

I also made a design decision that was not flashy but helped the whole team: I created stable test users and a demo roommate group. The seed data included demo users for Emma, Jerry, Thomas, Michelle, and Kelvin in a group with join code DEMO123, while keeping test@example.com independent from that group. That meant we could test two important paths every day: a user who already had household data and a user who still needed to create or join a group. The alternative was for everyone to create their own random users and groups manually. That would have made bugs harder to reproduce because one teammate might be testing with a grouped user while another was testing with an ungrouped user, and nobody would know whether a failure came from code or inconsistent data. The shared seed data became a lightweight version of the testing discipline from Lecture 9: not full TDD yet, but a repeatable setup that made manual testing more meaningful.

Clean Code Snippet

```
const submitGroupAction = async (url, body) => {  
  setError("");  
  setLoading(true);
```

```

try {
  const res = await fetch(url, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${accessToken}`,
    },
    body: JSON.stringify(body),
  });

  const data = await res.json();

  if (!res.ok) {
    setError(data.message || "Something went wrong.");
    return;
  }

  saveUserAndGoHome(data.user);
} catch (err) {
  setError("Network error. Please try again.");
  console.log(err);
} finally {
  setLoading(false);
}
};

```

This is the code I am proudest of because it shows a clean refactor from duplicated form logic into one reusable helper. Creating a group and joining a group are different user actions, but under the hood they do almost the same thing: prevent repeated loading issues, send a POST request, attach the JWT token, parse the backend response, show an error if something fails, and update the user before going back home. Instead of writing that same fetch logic twice, I pulled the shared behavior into `submitGroupAction`.

The above code is clean because its structure and function responsibilities are immediately evident. The helper function takes a URL and request body, submits the request, handles errors, and updates the user state on success. Meanwhile, `createGroup` and `joinGroup` only prepare the information unique to their respective actions before calling the helper. This makes the intent of each function obvious without requiring the reader to trace through large amounts of repeated logic. The control flow is also easy to follow, making it clear exactly what occurs at each step of the process.

This connects directly to the reuse and managing complexity lectures because the code reduces duplication without becoming too abstract. I believe it follows the Single Responsibility Principle particularly well. The responsibility of `submitGroupAction` is solely to handle the common workflow of submitting a group-related request and processing the result. In contrast, `createGroup` and `joinGroup` remain small and focused because they only describe what is unique about each action. `createGroup` determines the group name, while `joinGroup` determines the join code. This also demonstrates Separation of Concerns because the networking and request-handling logic is separated from the form-specific logic. If the request workflow ever changes, it only needs to be updated in one location rather than across multiple form handlers.

I also paid attention to naming conventions and avoiding magic values. Names such as `submitGroupAction`, `createGroup`, and `joinGroup` clearly communicate their purpose without requiring extensive comments. Similarly, constants such as `CREATE_GROUP_URL`, `JOIN_GROUP_URL`, and `DEFAULT_GROUP_NAME` make the code more self-documenting by replacing hardcoded strings with descriptive identifiers. This improves maintainability because changes can be made in a single location rather than throughout the codebase.

Another reason I like this snippet is that the control flow is easy to trust. The try/catch/finally structure makes the asynchronous behavior highly readable: attempt the request, handle any network or server errors, and always reset the loading state at the end. This acts as a form of defensive programming because the UI cannot become permanently stuck in a loading state if an error occurs. Future developers should not have to mentally untangle the function to understand what happens, and I think this implementation achieves that goal.

Overall, this refactor reflects several software construction principles discussed throughout the course, including reuse, separation of concerns, meaningful naming, defensive programming, and the Single Responsibility Principle. The final version is shorter, easier to read, easier to maintain, and more reliable than the duplicated implementation it replaced. I think this refactor is strong because it remains readable, reusable, and simple, which is exactly what good refactoring should accomplish.

Software Construction Principles Applied

I used a lot of software construction principles during this project, but the one that appeared most often was separation of concerns. Homiely became much easier to understand when each part of the system had a clear purpose. React pages handled the user interface, Flask routes handled API behavior, PostgreSQL handled persistent relational data, and utility files such as `balances.js` and `chores.js` handled repeated calculations. This was not only cleaner from a design perspective, but also very practical for a group project, because teammates could work in different areas without constantly colliding in the same files.

The Git lecture also changed how I thought about collaboration. Before this class, I mostly thought of Git as a way to save versions of code. During this project, I started treating it as the main tool for coordinating people. I encouraged separate branches, pulling from origin/main before merging, and keeping work in separate files when possible. When branches diverged or conflicts appeared, I used `git log --oneline` to understand where HEAD was, `git diff` to compare changes, and rebasing when it made sense to clean up branch history. This workflow was not perfect, but it gave us a way to keep moving without losing a stable version of the project.

The client-server lecture showed up constantly because almost every meaningful feature involved the same request-response pattern. React collected input and sent a fetch request. Flask received the request, checked authentication or request data, interacted with PostgreSQL, and returned JSON. Then React updated the UI based on that response. This made me much more aware of API contracts. For example, the login flow depended on the backend returning the user, tokens, and group membership state. If that response shape changed, frontend routing could break immediately.

React state also became a major construction principle in my work. The pages I worked on were not static screens; they reacted to product states such as loading, bad login, no group, grouped, editing profile, leaving a group, or waiting for dashboard statistics. I used `useState`, `useEffect`, dependency arrays, `async` fetch calls, and conditional JSX to make the UI match the actual state of the application. I started to think of the frontend like a small state machine, where a different state should lead to a different message, button, redirect, or dashboard.

Testing and debugging were another area where I learned a lot. We did not fully practice TDD, but the seed users and DEMO123 group gave the team repeatable manual test conditions. That mattered

because bugs became easier to reproduce when everyone knew the expected grouped and ungrouped states. When failures appeared, I learned to slow down and isolate the problem: whether Docker was running, whether Flask returned JSON, whether the token existed, whether the database was seeded, or whether React was rendering before the data existed. Console logs and backend errors were not fancy, but they helped identify the layer where the bug lived.

Security and authentication influenced the route-guard design. The security lecture made me think about confidentiality and integrity in a concrete way. A logged-out user should not be able to access private household pages by typing a URL, and a logged-in user should not keep seeing login/signup pages. Emma implemented the guard components, but I advocated for this design because it made authentication a consistent policy instead of scattered page checks.

Reuse and managing complexity influenced my refactors. The `submitGroupAction` helper reused the `fetch/error/loading` workflow for creating and joining groups, while the `Home` refactor reused `balance` and `chore` utility functions instead of hiding calculations inside JSX. I also organized frontend code into pages, components, routes, and CSS files so different teammates could work on different parts of the system. The main lesson I took from this is that clean code is not just code that looks nice; it is code where another person can quickly find where a behavior lives.

I also used GenAI during the project, but I tried to treat it as a coding partner rather than an authority. It was useful for brainstorming, debugging ideas, CSS help, and refactoring suggestions, especially when the work was tedious or repetitive. However, I still had to verify suggestions against the actual repository, routes, commits, and data model. That matched the GenAI lecture because AI can speed up development, but it does not remove the need for human review and project-specific judgment.

Finally, the code comprehension and code review lectures matched my experience in the project very closely. A lot of the work was not simply writing new code; it was reading teammates' code, understanding what a route or component did, checking whether a refactor changed behavior, and making sure different features still fit together. Reviewing diffs and communicating before merges made the project easier to understand and less risky to integrate.

Individual Contribution and Integration With Team Work

My individual contributions can be verified throughout the git history under TomLe1227 / 2005thomasle@gmail.com, with an early commit also under Thomas Le. My work was not limited to one isolated feature. I mainly worked on the pieces of the project that connected the app together: login routing, group setup, Home page rendering, profile membership behavior, Docker, seed data, database relationship decisions, frontend design, refactoring, and our team's Git workflow.

A large portion of my work was on the frontend. I worked heavily on the Landing page and the initial Home page, including the navbar, buttons, page layout, and visual polish. I also implemented visual features like the landing page background changing depending on the time of day, so the site could feel different during the day versus at night. These features were not only aesthetic; they helped make the project feel like a real product rather than a collection of disconnected class components. I also organized frontend files into pages, components, and CSS files so teammates could work independently on their own features without constantly editing the same files.

One of my larger contributions was the login and authentication flow on the frontend. I created the early login/signup file structure, worked on page navigation, and later integrated Emma's authentication work into my branch. In commit 2ae75f5, I implemented the two-case login behavior: if a user already belonged to a roommate group, they were routed to /home; if they did not belong to a group yet, they were routed to /group-setup. This made the login system feel connected to the actual state of the user instead of sending every user to the same page.

The authentication system also integrated into the frontend through JWTs. After login, the frontend stored the user and token information through AuthContext.jsx, and later requests sent the JWT in the Authorization: Bearer header. This mattered because pages such as Home, Chores, Expenses, Calendar, and Profile relied on the backend knowing which user was making the request. I also pushed for guarded routes because I wanted to make sure users could not manually type protected URLs and access private roommate data without being logged in. Emma implemented the final ProtectedRoutes and LoggedInProtectedRoutes components, but the idea came from trying to make authentication a real access-control boundary rather than just a login form.

Area	Evidence	Why it mattered
Frontend design and page structure	0db3caa, 3380ff5, 6b5838c, 4ee2357, 6047275, 0da6dca	I worked heavily on the Landing and Home frontend, including navbar/button design, page layout, visual polish, and the time-of-day background feature.
Login and routing	e3fa573, ab0aff5, 7e4529d, 2ae75f5	I helped build the login/signup structure, page navigation, and the two-case login flow where users with a group go to Home and users without a group go to GroupSetup.
Protected route idea	Emma implemented 60787d6 and a08cf9a	I pushed for a centralized route guard so private pages could not be reached just by typing a URL. This made authentication a real boundary instead of just a form.
Home and group state	0da6dca, d90b736, 0c616ef, ffa23c6, 3deb43d, 03c83d0	I built Home around real backend state, including the grouped dashboard, no-group empty state, join code, roommate count, chores, and expense summary.
Group setup/profile membership	2ae75f5, 582059b, 03c83d0	I worked on create/join group behavior, leave-group behavior, and keeping auth state, page state, and database membership consistent.
Postgres group/user relationship	models.py; RoommateGroupMember; group_routes.py	I helped keep users and groups independent by representing membership with user_id and group_id in a separate table.
Seed data and testing baseline	d90b736, 0c616ef, 519dd7b; backend/seed.py	I created the DEMO123 group and an independent test@example.com user so the team could test both grouped and ungrouped flows repeatedly.
Workflow/orchestration	Tommy/Michelle/Jerry/Kelvin branches, merge history	I coordinated separate branches, frontend file organization, file ownership, pulling origin/main, comparing diffs, and keeping a working version available.

I also contributed heavily to group membership behavior. I worked on the flow where users could create a group, join a group, leave a group, and then have the rest of the app update correctly afterward. This connected the frontend, Flask routes, auth context, and PostgreSQL database together. For example, when a user joined or left a group, the app needed to update both the backend membership data and the frontend user state so Home and Profile would render the correct information.

On the database side, I helped keep our group/user relationship modeled in a clean relational way. Instead of storing a group_id directly inside each user, the project used a separate RoommateGroupMember table to connect user_id and group_id. This was better practice because users and groups could remain independent records, while the relationship between them was stored separately. It also made testing easier because the independent test@example.com user could exist without belonging to the demo group, while the demo users could be linked to DEMO123 through membership rows.

Another contribution I made was creating stable seed data. I created a demo group with the join code DEMO123 and five demo users, while keeping test@example.com outside of that group. This gave the entire team a consistent testing setup. Teammates could test the grouped-user flow with the demo group, or test the new-user/no-group flow with the independent test account. This made bugs much easier to reproduce because we were not all testing with random local data.

Finally, I contributed to the team workflow itself. I pushed the group toward working on separate branches, pulling from origin/main before merging, and keeping features in separate files when possible. When merge issues happened, I used tools like git log --oneline, git diff, and rebasing to

understand what changed and how to recover. This workflow was important because our group had many features being developed at the same time, and organizing the work helped us avoid unnecessary conflicts.

Improvements and Additional Features

If I had more time, I would first improve our handling of group state. Conditional rendering worked well for the Home page, but over time Chores, Expenses, and Calendar also became dependent on whether a user belonged to a group. A GroupContext or GroupProtectedRoute would have made that dependency clearer and would have reduced repeated fetches and repeated conditional logic.

I would also improve our testing process. My seed users gave us repeatable manual testing, but they did not protect us from regressions by themselves. Looking back at Assignment 6, we could have used AI more intentionally to help us walk through a TDD-style workflow: write tests for expected behavior, run them, watch them fail, implement the feature, and then refactor once the tests passed. This would have been especially useful for login redirects, route guards, create group, join group, leave group, no-group Home rendering, and dashboard stats.

On the product side, I would also add notifications for upcoming chores, better expense settlement history, and a clearer onboarding flow for join codes. Those features would have made the app feel more complete for real roommate use, but the testing improvements would have mattered first because they would have made it safer to keep adding features without accidentally breaking existing flows.

Mistakes / What I Learned

The biggest mistake I made, in my opinion, was assuming that environment setup would be a small problem. At the start, it felt like each person could simply install the dependencies and run the frontend and backend locally. However, this quickly became one of the most frustrating parts of development. Different Python versions, Node versions, ports, and local database setups caused bugs that looked like application bugs, even when the actual code was fine. This wasted time because we were debugging machines instead of debugging Homiely.

The reason this went wrong was not that Docker was a bad choice. I still think Docker was the correct choice for our project. The real issue was that, as EE majors who were still new to building a larger full-stack software project, we underestimated how much setup, environment consistency, and shared testing mattered. We were learning not only React, Flask, and PostgreSQL, but also how to make a whole team run the same system consistently. This mistake taught me that setup is not a side detail; it is part of the software system.

Another mistake was relying too much on manual testing for too long. The demo users and DEMO123 group helped a lot, but they were only a baseline. They made bugs reproducible, but they did not automatically tell us when we broke something. A better approach would have been to use the AI-assisted TDD process we practiced in Assignment 6: describe the behavior, generate or write a focused test, run it, make it pass, and then refactor. Doing that for login redirects, group creation, joining groups, leaving groups, and no-group Home rendering would have given us more confidence when merging changes.

I also learned that a design can be reasonable early and still become messy later. Conditional rendering on Home was the right choice when I first implemented it because it was simple and intuitive.

However, once Chores, Expenses, and Calendar also needed to know whether a user was in a group, the same idea started spreading across multiple pages. That was a sign that we should have refactored into a shared GroupContext or GroupProtectedRoute. The mistake was not the original design; the mistake was waiting too long to centralize the pattern after the project grew.

Finally, I learned that team workflow has to be explicit. I communicated that we should work on separate branches, pull from main before merging, and avoid editing the same files when possible, but this process lived mostly in conversation. It should have been written down in a CONTRIBUTING file. If a workflow only exists in people's memory, it is easy for the group to drift away from it. In a future project, I would formalize the workflow earlier so the team has a shared contract rather than just shared habits.